🦀 Rust für C/C++ Entwickler

Sicher. Schnell. Ohne Garbage Collector.

> „Rust ist C++ – aber der Compiler übernimmt das, woran dein Kollege immer gescheitert ist."

Die Geschichte von Rust

2006 – Der Aufzug: Graydon Hoare kommt nach Hause. Der Aufzug ist wegen eines Software-Absturzes ausgefallen. Er wohnt im **21. Stock.** Auf dem Treppensteigen denkt er: *„Wir können keinen Aufzug bauen, der nicht abstürzt?"* – Er öffnet seinen Laptop und beginnt Rust zu schreiben.

Jahr	Meilenstein
2009	Mozilla sponsert Rust offiziell
2013	Garbage Collector entfernt – Rust läuft ohne GC
2015	**Rust 1.0** – erste stabile Version
2016–2022	7× in Folge beliebteste Sprache (Stack Overflow Survey)
2021	Rust Foundation gegründet (Mozilla, Google, Microsoft, Amazon, …)
2022	Linux-Kernel nimmt Rust als zweite Systemsprache auf

Warum Rust?

	C/C++	Rust
Performance	✅	✅
Memory Safety	❌ manuell	✅ compile-time
Data Races	❌	✅ unmöglich
Garbage Collector	❌	❌
Zero-Cost Abstraktion	✅	✅
Tooling	😬	✅ cargo

> Kein Runtime-Overhead. Kein GC.

Zero-Cost Abstraktion

> „What you don't use, you don't pay for. What you do use, you couldn't write better by hand." — Bjarne Stroustrup

Hochlesbarer Code — **identisches Assembly** wie die handgeschriebene Schleife:

```rust
let sum: i32 = vec![1, 2, 3, 4, 5]
    .iter()
    .filter(|&&x| x % 2 == 0)
    .map(|&x| x * x)
    .sum();
```

```c
// Äquivalentes C — der Compiler erzeugt denselben Maschinencode
int sum = 0;
for (int i = 0; i < 5; i++)
    if (arr[i] % 2 == 0)
        sum += arr[i] * arr[i];
```

Cargo & Tooling

C/C++: `cmake`, `make`, `vcpkg`, `conan`, `gdb`, `valgrind` …

Rust: Einfach `cargo`.

```
cargo new my_project     # Projekt erstellen
cargo build              # Kompilieren
cargo run                # Bauen + Ausführen
cargo test               # Tests ausführen  ← built-in, kein Framework!
cargo add serde          # Dependency hinzufügen
cargo clippy             # Linter
cargo fmt                # Auto-Formatter
cargo doc --open         # Dokumentation generieren
```

> `Cargo.toml` = `CMakeLists.txt` + Paketmanager + Test-Runner in einem

Ownership – Das Kernkonzept

📂 `examples/01_ownership/main.rs`

In C/C++: **Du** bist verantwortlich für Speicher.

```
int* ptr = new int(42);
use(ptr);
delete ptr;       // Vergiss das → Memory Leak
use(ptr);         // Use-after-free → 💥
```

In Rust: **Der Compiler** überwacht den Besitz.

```
let s = String::from("hello"); // s besitzt den Speicher
use_string(s);                 // Ownership wird übergeben (move)
println!("{}", s);             // ❌ COMPILE ERROR: s wurde moved
                               // kein delete nötig – automatisch gedroppt
```

> **Regel:** Jeder Wert hat genau **einen** Owner. Endet der Scope → `free()`.

Move Semantics

📂 `examples/01_ownership/main.rs`

```cpp
// C++: Move ist opt-in — vergisst man leicht
std::vector<int> a = {1, 2, 3};
std::vector<int> b = std::move(a);
a.push_back(4);  // UB: "valid but unspecified state" 💥
```

```rust
// Rust: Move ist der Default — Compiler verhindert Missbrauch
let a = vec![1, 2, 3];
let b = a;        // impliziter Move — a ist weg
a.push(4);        // ❌ COMPILE ERROR — kein UB, kein Crash
```

```rust
// Explizite Kopie: clone()
let a = vec![1, 2, 3];
let b = a.clone(); // deep copy
println!("{:?} {:?}", a, b); // ✅ beide gültig
```

Borrowing – Referenzen ohne Gefahr

📂 `examples/02_borrowing/main.rs`

```rust
fn main() {
    let mut s = String::from("hello");

    let r1 = &s;      // ✅ immutable borrow
    let r2 = &s;      // ✅ N immutable borrows erlaubt
    println!("{} {}", r1, r2);
    // r1, r2 nicht mehr genutzt → Borrows enden hier

    let r3 = &mut s;  // ✅ genau 1 mutable borrow
    r3.push_str(", world");
    println!("{}", r3);
}
```

> **Borrowing-Regeln** (zur Compile-Zeit geprüft – kein Runtime-Overhead):
>
> - Entweder **beliebig viele** `&T` (immutable)
> - Oder **genau eine** `&mut T` (mutable)
>
> – nie beides gleichzeitig → **keine Data Races möglich**

Lifetimes

📂 `examples/03_lifetimes/main.rs`

```rust
// Welche Referenz wird zurückgegeben? Der Compiler muss es wissen.
fn longer<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
//  'a: Rückgabewert lebt so lange wie das KÜRZERE der beiden
}
```

```rust
fn main() {
    let s1 = String::from("long string");
    let result;
    {
        let s2 = String::from("xyz");
        result = longer(s1.as_str(), s2.as_str());
        println!("{}", result); // ✅
    }
    // println!("{}", result); // ❌ s2 lebt nicht mehr → Compile Error
}
```

> `'a` ist **kein Runtime-Konzept** – reine Compiler-Annotation.
> Meist inferiert der Compiler Lifetimes automatisch.

Error Handling – Result<T, E>

examples/04_error_handling/main.rs

```rust
use std::fs::File;

fn main() {
    // Result<T,E> zwingt zur Behandlung – kein silent ignore
    match File::open("data.txt") {
        Ok(file) => println!("Geöffnet: {:?}", file),
        Err(e)   => println!("Fehler: {}", e),
    }
}
```

```rust
// Der ? Operator: Fehler propagieren ohne Boilerplate
fn read_username() -> Result<String, std::io::Error> {
    let s = std::fs::read_to_string("user.txt")?;
    //                                          ^ Err → early return
    Ok(s.trim().to_string())
}
```

`Option<T>` analog – für nullable Werte statt Fehler (`Some` / `None`).

Traits & Generics

examples/05_traits/main.rs

```rust
trait Area { fn area(&self) -> f64; }

struct Circle    { radius: f64 }
struct Rectangle { width: f64, height: f64 }

impl Area for Circle    { fn area(&self) -> f64 { std::f64::consts::PI * self.radius * self.radius } }
impl Area for Rectangle { fn area(&self) -> f64 { self.width * self.height } }

// impl Trait → statischer Dispatch (Zero-Cost)
fn print_static(shape: &impl Area)  { println!("static:  {:.2}", shape.area()); }

// dyn Trait → dynamischer Dispatch (vtable)
fn print_dynamic(shape: &dyn Area)  { println!("dynamic: {:.2}", shape.area()); }

fn main() {
    let shapes: Vec<Box<dyn Area>> = vec![
        Box::new(Circle { radius: 3.0 }),
        Box::new(Rectangle { width: 4.0, height: 5.0 }),
    ];
    for s in &shapes { print_dynamic(s.as_ref()); }
}
```

🗺️ Wo wird Rust eingesetzt?

CLI	Web	WASM	Embedded	Desktop
ripgrep	axum	Figma	Linux Kernel	Zed
bat	Discord	Google Earth	Android	Tauri
fd	Cloudflare Workers	Web UI (Leptos)	Infineon	Gitbutler
exa	npm Registry	Dioxus	STM32/ESP	Spacedrive

> Rust läuft überall – vom Mikrocontroller bis zum Cloud-Datacenter.

DEMO 1 — CLI Tool

`demos/01_cli/cli.rs`

```rust
use std::{env, fs, process};

fn main() {
    let args: Vec<String> = env::args().collect();
    if args.len() < 3 {
        eprintln!("Usage: {} <query> <file>", args[0]);
        process::exit(1);
    }

    let contents = fs::read_to_string(&args[2]).unwrap_or_else(|e| {
        eprintln!("Fehler: {}", e); process::exit(1);
    });

    for (num, line) in search(&args[1], &contents) {
        println!("{:>4}: {}", num, line);
    }
}

// Zero-Copy: Slices zeigen in die Originaldaten — keine Heap-Allokation
fn search<'a>(query: &str, contents: &'a str) -> Vec<(usize, &'a str)> {
    contents.lines().enumerate()
        .filter(|(_, l)| l.contains(query))
        .map(|(i, l)| (i + 1, l))
        .collect()
}
```

> Echte Projekte: **ripgrep**, bat, fd, exa, delta, zoxide

DEMO 2 – Web Server (axum)

`demos/02_webserver/src/main.rs`

```rust
#[derive(Serialize, Deserialize, Clone)]
struct User { id: u32, name: String, age: u32 }

type SharedState = Arc<Mutex<Vec<User>>>;

#[tokio::main]
async fn main() {
    let state: SharedState = Arc::new(Mutex::new(vec![...]));
    let app = Router::new()
        .route("/users",     get(get_users).post(create_user))
        .route("/users/:id", get(get_user_by_id))
        .with_state(state);
    axum::serve(TcpListener::bind("0.0.0.0:3000").await.unwrap(), app).await.unwrap();
}

async fn get_user_by_id(State(s): State<SharedState>, Path(id): Path<u32>)
    -> Result<Json<User>, StatusCode> {
    s.lock().unwrap().iter().find(|u| u.id == id)
        .cloned().map(Json).ok_or(StatusCode::NOT_FOUND)
}
```

```
curl http://localhost:3000/users
curl -X POST http://localhost:3000/users -d '{"name":"Alice","age":30}'
```

> Echte Projekte: **Discord**, Cloudflare Workers, npm Registry

DEMO 3 — Systems / Performance

`demos/03_systems/systems.rs`

```rust
// Zero-Copy Record: Slices zeigen in die Originaldaten
#[derive(Debug)]
struct CsvRecord<'a> { name: &'a str, age: u32, city: &'a str, salary: f64 }

fn parse_record(line: &str) -> Option<CsvRecord<'_>> {
    let mut f = line.splitn(4, ',');
    Some(CsvRecord {
        name:   f.next()?.trim(),
        age:    f.next()?.trim().parse().ok()?,
        city:   f.next()?.trim(),
        salary: f.next()?.trim().parse().ok()?,
    })
}

let start = Instant::now();
let records: Vec<CsvRecord> = csv.lines().skip(1).filter_map(parse_record).collect();
println!("{} Records in {:.2?}", records.len(), start.elapsed());

// FFI: C-Stdlib direkt aufrufen — zero overhead
extern "C" { fn abs(x: i32) -> i32; }
let result = unsafe { abs(-42) };
```

> Echte Projekte: **ripgrep**, rust-analyzer, SWC, Turbopack, Deno

DEMO 4 – WebAssembly

`demos/04_wasm/wasm.rs`

```
rustup target add wasm32-unknown-unknown
cargo build --target wasm32-unknown-unknown --release
wasm-bindgen --out-dir ./out --target web target/.../pkg.wasm
```

```rust
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn fibonacci(n: u32) -> u64 { /* ... */ }

// Rust-Struct wird zur JavaScript-Klasse
#[wasm_bindgen]
pub struct Counter { value: i32, step: i32 }

#[wasm_bindgen]
impl Counter {
    #[wasm_bindgen(constructor)]
    pub fn new(step: i32) -> Counter { Counter { value: 0, step } }
    pub fn increment(&mut self) { self.value += self.step; }
    pub fn get(&self) -> i32    { self.value }
}
```

```javascript
// JavaScript – native Typen, keine Wrapper
const c = new Counter(5);
c.increment();
console.log(c.get(), fibonacci(10)); // → 5, 55
```

> Echte Projekte: **Figma** (Rendering Engine), Google Earth, Cloudflare Workers

DEMO 5 – Embedded / no_std

`demos/05_embedded/embedded.rs`

```
rustup target add thumbv7em-none-eabihf
cargo build --target thumbv7em-none-eabihf
probe-rs run --chip STM32F411RETx
```

```rust
#![no_std]   // kein std, kein Heap, kein OS
#![no_main]

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    let p = embassy_stm32::init(Default::default());

    // Typsicher: falscher Pin → Compile Error, nicht Runtime Error
    let mut led = Output::new(p.PB7, Level::High, Speed::Low);

    // Zweiter Task – kein FreeRTOS, kein RTOS nötig!
    spawner.spawn(blink_fast(led2)).unwrap();

    loop {
        led.set_high();
        Timer::after(Duration::from_millis(500)).await; // async – kein Busy-Wait!
        led.set_low();
        Timer::after(Duration::from_millis(500)).await;
    }
}
```

> Echte Projekte: **Linux-Kernel** (seit 6.1!), Android (Binder), Azure Sphere, Infineon Automotive

DEMO 6 – Web UI (Leptos)

`demos/06_webui/webui.rs` · `cargo install trunk && trunk serve`

```rust
use leptos::*;

// Reaktive Komponente – kompiliert zu WASM, kein JS-Framework nötig
#[component]
fn Counter() -> impl IntoView {
    let (count, set_count) = create_signal(0);
    let doubled = move || count() * 2;

    view! {
        <div>
            <p>"Count: " {count} " (doubled: " {doubled} ")"</p>
            <button on:click=move |_| set_count.update(|n| *n -= 1)>"-"</button>
            <button on:click=move |_| set_count.set(0)>"Reset"</button>
            <button on:click=move |_| set_count.update(|n| *n += 1)>"+"</button>
        </div>
    }
}

fn main() { mount_to_body(|| view! { <Counter /> }) }
```

> Kein Virtual DOM. Direktes WASM → DOM Update.
> Template-Fehler sind **Compile-Fehler**.
> Echte Projekte: **Zed** (eigenes GPU-UI-Framework), Leptos, Dioxus

DEMO 7 — Desktop App (Tauri)

📂 `demos/07_desktop/desktop.rs` · `cargo tauri dev`

```rust
#[derive(Serialize, Deserialize, Clone)]
struct Note { id: u32, title: String, content: String }

// Rust-Funktionen direkt aus JavaScript aufrufbar — typsicher
#[command]
fn get_notes(state: State<AppState>) -> Vec<Note> {
    state.notes.lock().unwrap().clone()
}

#[command]
fn add_note(title: String, content: String, state: State<AppState>) -> Note { /* ... */ }
```

```rust
// JavaScript Frontend (React / Svelte / Vanilla):
import { invoke } from '@tauri-apps/api/tauri';
const notes = await invoke('get_notes');
const note  = await invoke('add_note', { title: 'Hey', content: '🦀' });
```

	Electron	Tauri
Bundle-Grösse	~150 MB	~8 MB
RAM-Verbrauch	~200 MB	~30 MB

> Echte Projekte: **Zed**, Gitbutler, Spacedrive

Zusammenfassung

Konzept	Rust-Lösung
Memory Safety	Ownership + Borrow Checker (compile-time)
Fehlerbehandlung	`Result<T,E>` + `?` Operator
Abstraktion	Traits – statisch & dynamisch (Zero-Cost)
Tooling	`cargo` – alles in einem
Einsatzgebiete	CLI · Web · WASM · Embedded · Desktop · UI

🦀 Wo anfangen?

```
# Rust installieren
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Interaktiv lernen – kleine Übungsaufgaben im Terminal
cargo install rustlings && rustlings

# VSCode Extension
rust-analyzer
```

Lesen:

- 📖 The Rust Book – kostenlos online
- 📖 Programming Rust – O'Reilly, ideal für C++-Hintergrund
- 📖 Rustonomicon – für unsafe Rust

Fragen? 🙋